

# CS336 Assignment 1: Basics

Writeup

## Contents

|                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------|----|
| 1. Problem ( <code>unicode1</code> ): Understanding Unicode (1 point) .....                                  | 2  |
| 2. Problem ( <code>unicode2</code> ): Unicode Encodings (3 points) .....                                     | 2  |
| 3. Problem ( <code>train_bpe_tinystories</code> ): BPE Training on TinyStories (2 points) .....              | 3  |
| 4. Problem ( <code>train_bpe_expts_owt</code> ): BPE Training on OpenWebText (2 points) .....                | 4  |
| 5. Problem ( <code>tokenizer_experiments</code> ): Experiments with tokenizers (4 points) .....              | 4  |
| 6. Problem ( <code>transformer_accounting</code> ): Transformer LM resource accounting (5 points) .....      | 5  |
| 7. Problem ( <code>learning_rate_tuning</code> ): Tuning the learning rate (1 point) .....                   | 8  |
| 8. Problem ( <code>adamw_accounting</code> ): Resource accounting for training with AdamW (2 points) .....   | 8  |
| 9. Problem ( <code>learning_rate</code> ): Tune the learning rate (2 B200 hrs) (3 points) .....              | 12 |
| 10. Problem ( <code>batch_size_experiment</code> ): Batch size variations (1 B200 hr) (1 point) .....        | 17 |
| 11. Problem ( <code>generate</code> ): Generate text (1 point) .....                                         | 17 |
| 12. Problem ( <code>layer_norm_ablation</code> ): Remove RMSNorm and train (0.5 B200 hrs) (1 point) .....    | 17 |
| 13. Problem ( <code>pre_norm_ablation</code> ): Implement post-norm and train (0.5 B200 hrs) (1 point) ..... | 18 |
| 14. Problem ( <code>no_pos_emb</code> ): Implement NoPE (0.5 B200 hrs) (1 point) .....                       | 18 |
| 15. Problem ( <code>swiglu_ablation</code> ): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point) .....                 | 18 |
| 16. Problem ( <code>main_experiment</code> ): Experiment on OWT (2 B200 hrs) (2 points) .....                | 19 |
| 17. Problem ( <code>leaderboard</code> ): Leaderboard (10 B200 hrs) (6 points) .....                         | 19 |

## 1. Problem (unicode1): Understanding Unicode (1 point)

### Prompt

(a) What Unicode character does `chr(0)` return?

**Deliverable:** A one-sentence response.

(b) How does this character's string representation (`__repr__()`) differ from its printed representation?

**Deliverable:** A one-sentence response.

(c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

**Deliverable:** A one-sentence response.

### Answer

(a) 返回的是码点为 U+0000，名称为 NUL 的控制字符，解释器以 `'\x00'` 这种转义表示来显示它。

(b) `__repr__()` 用转义的可见文本描述字符，`print` 输出实际的字符，NUL 字符通常没有可见字形，其 `print` 结果通常不可见。

(c) 在交互式解释器直接输入表达式时，解释器自动显示其 `repr`，而 `print` 写出字符串的实际内容，像 `"this is a test" + chr(0) + "string"` 里 `chr(0)` 这个字符插在文本中，只是打印时不可见。

```
>>> chr(0)
'\x00'
>>> print(chr(0))

>>> "this is a test" + chr(0) + "string"
'this is a test\x00string'
>>> print("this is a test" + chr(0) + "string")
this is a teststring
```

## 2. Problem (unicode2): Unicode Encodings (3 points)

### Prompt

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

**Deliverable:** A one-to-two sentence response.

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
```

```
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))  
'hello'
```

**Deliverable:** An example input byte string for which `decode_utf8_bytes_to_str_wrong` produces incorrect output, with a one-sentence explanation of why the function is incorrect.

(c) Give a two-byte sequence that does not decode to any Unicode character(s).

**Deliverable:** An example, with a one-sentence explanation.

### Answer

(a) UTF-8 是变长编码，而且是当前很多网页都在使用的主要编码。相较 UTF-16 和 UTF-32 更加省空间，并且 ASCII 文本不会像 UTF-16/UTF-32 那样出现大量填充零字节。

(b) 因为 UTF-8 是变长的，而当前函数是尝试对每个输入的字节都进行单独 `decode`，所以会出现问题。一个错误示例是“牛”，其 utf-8 表示为 `b'\xe7\x89\x9b'`，在调用本函数时会出现 `UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 0: unexpected end of data` 的报错。

(c) 对于 UTF-8 来说：起始字节的高位 1 的个数表示总字节数，后续所有字节必须严格以 10 开头。所以一个错误例子可以是 `11100111 10001001` (`b'\xe7\x89'`)，这里第一位告诉要 3 个字节，但只给了两个字节。

## 3. Problem (`train_bpe_tinystories`): BPE Training on TinyStories (2 points)

### Prompt

(a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How much time and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements:  $\leq 30$  minutes (no GPUs),  $\leq 30$  GB RAM

Hint: You should be able to get under 2 minutes for BPE training using multiprocessing during pre-tokenization and the following two facts:

(a) The `<|endoftext|>` token delimits documents in the data files.

(b) The `<|endoftext|>` token is handled as a special case before the BPE merges are applied.

**Deliverable:** A one-to-two sentence response.

(b) Profile your code. What part of the tokenizer training process takes the most time?

**Deliverable:** A one-to-two sentence response.

### Answer

(a) 训练的 wall-clock 时间约为 21.23 秒；GNU `time -v` 报告的峰值 RSS 为 188.5 MiB。最长的 token 是 `'accomplishment'`，它是一个完整且在儿童故事中常见的英文词，前导空格也符合 GPT-2 风格预分词会把空格并入词元的行为。

(b) 本次训练最耗时的阶段是 pre-tokenization; 在 worker 内, 反复从 regex match 中提取完整匹配文本的 Match.group() 调用占据了主要时间。

#### 4. Problem (train\_bpe\_expts\_owt): BPE Training on OpenWebText (2 points)

##### Prompt

(a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements:  $\leq 12$  hours (no GPUs),  $\leq 100$  GB RAM

**Deliverable:** A one-to-two sentence response.

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

**Deliverable:** A one-to-two sentence response.

##### Answer

(a) 最长的 token 是一个 64 字节的序列, 由重复 16 次的 c383c382 组成, 显示为 ÅÅÅÅ...。原始 OpenWebText 数据中确实存在相同的编码伪影, 是网页语料中的乱码; 虽然没有语义意义, 但对该语料来说是合理的。

(b) TinyStories tokenizer 主要学习干净、可读的英文词元, 例如 b' accomplishment'; OpenWebText 的词表更大、内容更杂, 会学习网页格式、罕见字符串以及编码伪影等 token。

#### 5. Problem (tokenizer\_experiments): Experiments with tokenizers (4 points)

##### Prompt

(a) Sample 10 documents from TinyStories and OpenWebText. Using your previously trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary sizes, respectively), encode these sampled documents into integer ids. What is each tokenizer's compression ratio, measured in bytes per token?

**Deliverable:** A one-to-two sentence response.

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

**Deliverable:** A one-to-two sentence response.

(c) Estimate the throughput of your tokenizer, for example in bytes per second. About how long would it take to tokenize the Pile dataset (825 GB of text)?

**Deliverable:** A one-to-two sentence response.

(d) Using your TinyStories and OpenWebText tokenizers, encode the corresponding training and development datasets into sequences of integer token ids. We recommend storing the token ids as a NumPy array with dtype uint16. Why is uint16 an appropriate choice?

**Deliverable:** A one-to-two sentence response.

### Answer

(a) 在 TinyStories 数据集上面, TinyStories Tokenizer 的压缩比为 4.22, OWT Tokenizer 的压缩比为 4.09。

(b) 在 OWT 数据集上面, TinyStories Tokenizer 的压缩比为 3.21, OWT Tokenizer 的压缩比为 4.26。说明在 OWT 样本上, OWT tokenizer 的 bytes/token 高, 表示同样字节数被编码成更少 token, 压缩更好。

(c) TinyStories Tokenizer 的吞吐大概是 5853714 bytes/second, 大约 5.58 MiB/s, 训练 Pile dataset(825GiB, 搜了一下是这个大小) 需要大约 42 小时; 而 OWT Tokenizer 的吞吐大概是 4696115 bytes/second, 大约 4.47 MiB/s, 训练 Pile dataset 需要大约 52 小时

(d) 因为两个词表大小分别为 32K 和 10K, 用 uint16 足以覆盖两者所有的 token ID, 并且比再往后的 uint32, uint64 更省空间。

## 6. Problem (transformer\_accounting): Transformer LM resource accounting (5 points)

### Prompt

(a) Consider a GPT-2 XL-sized model using our assignment architecture, which has the following configuration:

- vocab\_size: 50,257
- context\_length: 1,024
- num\_layers: 48
- d\_model: 1,600
- num\_heads: 25
- d\_ff: 4,288 (the nearest multiple of 64 to  $8 / 3 \times 1,600$ )

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

**Deliverable:** A one-to-two sentence response.

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that your input sequence has context\_length tokens.

**Deliverable:** A list of matrix multiplies, with descriptions, and the total number of FLOPs required.

(c) Based on your analysis above, which parts of the model require the most FLOPs?

**Deliverable:** A one-to-two sentence response.

(d) Repeat your analysis with GPT-2 small (12 layers, 768 d\_model, 12 heads), GPT-2 medium (24 layers, 1024 d\_model, 16 heads), and GPT-2 large (36 layers, 1280 d\_model, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

**Deliverable:** For each model, provide a breakdown of model components and their associated FLOPs as a proportion of the total FLOPs required for a forward pass. In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

(e) Take GPT-2 XL and increase the context length to **16,384**. How does the total FLOPs for one forward pass change? How does the relative contribution of FLOPs of the model components change?

**Deliverable:** A one-to-two sentence response.

## Answer

assume  $B = \text{batch\_size}, T = \text{context\_length}, L = \text{num\_layers}, H = \text{num\_heads}, V = \text{vocab\_size}, d = d_{\text{model}}$

对于我们的模型，可训练的参数为  $d \times (1 + 2 \times V + 2 \times L + 4 \times L \times d + 3 \times L \times d_{\text{ff}})$

总 FLOPs 为  $2 \times T \times d \times (V + 2 \times T \times L + 4 \times L \times d + 3 \times L \times d_{\text{ff}})$

$$\frac{\text{FLOPS}}{\text{PARAMETERS}} = 2 \times T \times \frac{V + 2 \times T \times L + 4 \times L \times d + 3 \times L \times d_{\text{ff}}}{1 + 2 \times V + 2 \times L + 4 \times L \times d + 3 \times L \times d_{\text{ff}}}$$

(a) GPT-2 XL has 1,640,452,800 trainable parameters, which is 6,561,811,200 bytes, about 6.11 GiB.

(b) assuming batch size 1, GPT-2 XL has 3,516,769,894,400 FLOPs.

Total: 3,516,769,894,400 FLOPs

- | - layers: 3,352,087,756,800 FLOPs (95.32%)
    - | - attn: 1,328,755,507,200 FLOPs (37.78%)
      - | - q\_proj: 251,658,240,000 FLOPs (7.16%)
      - | - k\_proj: 251,658,240,000 FLOPs (7.16%)
      - | - v\_proj: 251,658,240,000 FLOPs (7.16%)
      - | - output\_proj: 251,658,240,000 FLOPs (7.16%)
      - | -  $QK^T$ : 161,061,273,600 FLOPs (4.58%)
      - | -  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ : 161,061,273,600 FLOPs (4.58%)
    - | - FFN: 2,023,332,249,600 FLOPs (57.53%)
      - | - w1: 674,444,083,200 FLOPs (19.18%)
      - | - w2: 674,444,083,200 FLOPs (19.18%)
      - | - w3: 674,444,083,200 FLOPs (19.18%)
  - | - LM head: 164,682,137,600 FLOPs (4.68%)
- (c) The FFN requires the most FLOPs.

(d) 可以注意到，随着参数量的增长，Transformer blocks 总占比从 72.90% 升至 92.55%；attention 从 33.13% 升至 38.25%，FFN 从 39.76% 升至 54.30%；LM head 从 27.10% 降至 7.45%。

GPT-2 small has 291,648,307,200 FLOPs

Total: 291,648,307,200 FLOPs

- | - layers: 212,600,881,152 FLOPs (72.90%)
  - | - attn: 96,636,764,160 FLOPs (33.13%)
    - | - q\_proj: 14,495,514,624 FLOPs (4.97%)
    - | - k\_proj: 14,495,514,624 FLOPs (4.97%)
    - | - v\_proj: 14,495,514,624 FLOPs (4.97%)
    - | - output\_proj: 14,495,514,624 FLOPs (4.97%)
    - | -  $QK^T$ : 19,327,352,832 FLOPs (6.63%)

- | -  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ : 19,327,352,832 FLOPs (6.63%)
- | - FFN: 115,964,116,992 FLOPs (39.76%)
  - | - w1: 38,654,705,664 FLOPs (13.25%)
  - | - w2: 38,654,705,664 FLOPs (13.25%)
  - | - w3: 38,654,705,664 FLOPs (13.25%)
- | - LM head: 79,047,426,048 FLOPs (27.10%)

GPT-2 medium has 830,172,299,264 FLOPs

Total: 830,172,299,264 FLOPs

- | - layers: 724,775,731,200 FLOPs (87.30%)
  - | - attn: 309,237,645,312 FLOPs (37.25%)
    - | - q\_proj: 51,539,607,552 FLOPs (6.21%)
    - | - k\_proj: 51,539,607,552 FLOPs (6.21%)
    - | - v\_proj: 51,539,607,552 FLOPs (6.21%)
    - | - output\_proj: 51,539,607,552 FLOPs (6.21%)
    - | -  $QK^T$ : 51,539,607,552 FLOPs (6.21%)
    - | -  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ : 51,539,607,552 FLOPs (6.21%)
  - | - FFN: 415,538,085,888 FLOPs (50.05%)
    - | - w1: 138,512,695,296 FLOPs (16.68%)
    - | - w2: 138,512,695,296 FLOPs (16.68%)
    - | - w3: 138,512,695,296 FLOPs (16.68%)
- | - LM head: 105,396,568,064 FLOPs (12.70%)

GPT-2 large has 1,768,530,903,040 FLOPs

Total: 1,768,530,903,040 FLOPs

- | - layers: 1,636,785,192,960 FLOPs (92.55%)
  - | - attn: 676,457,349,120 FLOPs (38.25%)
    - | - q\_proj: 120,795,955,200 FLOPs (6.83%)
    - | - k\_proj: 120,795,955,200 FLOPs (6.83%)
    - | - v\_proj: 120,795,955,200 FLOPs (6.83%)
    - | - output\_proj: 120,795,955,200 FLOPs (6.83%)
    - | -  $QK^T$ : 96,636,764,160 FLOPs (5.46%)
    - | -  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ : 96,636,764,160 FLOPs (5.46%)
  - | - FFN: 960,327,843,840 FLOPs (54.30%)
    - | - w1: 320,109,281,280 FLOPs (18.10%)
    - | - w2: 320,109,281,280 FLOPs (18.10%)
    - | - w3: 320,109,281,280 FLOPs (18.10%)
- | - LM head: 131,745,710,080 FLOPs (7.45%)

(e) 总 FLOPs 从  $3.52e12$  增加到  $1.34e14$ , 约增加 38 倍。attention 从 37.78% 升至 73.79%; FFN 从 57.53% 降至 24.24%; LM head 从 4.68% 降至 1.97%。其中 attention 部分的比重很高, 主要是  $QK^T$  和  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Total: 133,577,729,638,400 FLOPs

- | - layers: 130,942,815,436,800 FLOPs (98.03%)
  - | - attn: 98,569,499,443,200 FLOPs (73.79%)
    - | - q\_proj: 4,026,531,840,000 FLOPs (3.01%)
    - | - k\_proj: 4,026,531,840,000 FLOPs (3.01%)
    - | - v\_proj: 4,026,531,840,000 FLOPs (3.01%)
    - | - output\_proj: 4,026,531,840,000 FLOPs (3.01%)
    - | -  $QK^T$ : 41,231,686,041,600 FLOPs (30.87%)

- | -  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ : 41,231,686,041,600 FLOPs (30.87%)
- | - FFN: 32,373,315,993,600 FLOPs (24.24%)
- | - w1: 10,791,105,331,200 FLOPs (8.08%)
- | - w2: 10,791,105,331,200 FLOPs (8.08%)
- | - w3: 10,791,105,331,200 FLOPs (8.08%)
- | - LM head: 2,634,914,201,600 FLOPs (1.97%)

## 7. Problem (learning\_rate\_tuning): Tuning the learning rate (1 point)

### Prompt

As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example.

Run the SGD example above with three other values for the learning rate, `1e1`, `1e2`, and `1e3`, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge, that is, increase over the course of training?

**Deliverable:** A one-to-two sentence response with the behaviors you observed.

### Answer

对于相同的初始 weights:

- 1e1: loss 从 27.13 降到 3.64
- 1e2: loss 从 27.13 降到 6.32e-23
- 1e3: loss 从 27.13 升到 2.51e18

可以看到, 学习率从 1e1 改到 1e2 时 loss 下降地更快, 而 1e3 时训练发散

## 8. Problem (adamw\_accounting): Resource accounting for training with AdamW (2 points)

### Prompt

Let us compute how much memory and compute running AdamW requires. Assume we are using `float32` for every tensor.

(a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of `batch_size` and the model hyperparameters (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`). Assume `d_ff = 8 / 3 × d_model`.

For simplicity, when calculating memory usage of activations, consider only the following components:

- Transformer block: RMSNorm(s)
- Transformer block: multi-head self-attention sublayer (QKV projections,  $QK^T$  matrix multiply, softmax, weighted sum of values, output projection)
- Transformer block: position-wise feed-forward (SwiGLU): w1, w2, SiLU on the gate branch, element-wise product, w3
- Final RMSNorm
- Output embedding

- Cross-entropy on logits

**Deliverable:** An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on `batch_size`. What is the maximum batch size you can use and still fit within 80 GB memory?

**Deliverable:** An expression that looks like `a * batch_size + b` for numerical values `a`, `b`, and a number representing the maximum batch size.

(c) How many FLOPs does running one step of AdamW take?

**Deliverable:** An algebraic expression, with a brief justification.

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput, tokens per second, relative to the hardware's theoretical peak FLOP throughput [A. Chowdhery et al., 2022]. An NVIDIA H100 GPU has a theoretical peak of 495 **teraFLOP/s** for “float32”, actually TensorFlow-32, which in reality is “bfloat19”, operations. Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single H100? Following J. Kaplan et al. [25] and J. Hoffmann et al. [26], assume that the backward pass has twice the FLOPs of the forward pass.

**Deliverable:** The number of hours training would take, with a brief justification.

## Answer

(a)

Transformer LM

assume  $B = \text{batch\_size}$ ,  $T = \text{context\_length}$ ,  $L = \text{num\_layers}$ ,  $H = \text{num\_heads}$ ,  $V = \text{vocab\_size}$ ,  $d = d_{\text{model}}$

- Token Embedding
  - Parameter:  $V \times d$
- $L \times$  Transformer Block
  - Total:
    - Parameter:  $2 \times L \times d \times (1 + 6 \times d)$
    - Activation:  $\frac{2}{3} \times B \times T \times L \times (3 + 3 \times T \times H + 28 \times d)$
  - Each Part:
    - RMSNorm(ln1)
      - Parameter:  $d$
      - Activation:
        - rms:  $\text{batch\_size context\_length } 1$ 
          - $B \times T$
        - output:  $\text{batch\_size context\_length } d_{\text{model}}$ 
          - $B \times T \times d$
    - Causal Multi-Head Self-Attention (attn)
      - QKV projections
        - Parameter:  $3 \times d^2$
        - Activation:
          - $\text{batch\_size context\_length } d_{\text{model}} \rightarrow \text{batch\_size context\_length } d_{\text{model}}$
          - $3 \times B \times T \times d$
      - $Q \times K^T$  matrix multiply

- Activation:
    - [batch\_size context\_length d\_model] [batch\_size context\_length d\_model] -> batch\_size num\_heads context\_length context\_length
    - $B \times H \times T \times T$
- softmax
  - Activation:
    - batch\_size num\_heads context\_length context\_length
    - $B \times H \times T \times T$
- weighted sum of values
  - Activation:
    - batch\_size context\_length d\_model
    - $B \times T \times d$
- output projections
  - Parameter:  $d^2$
  - Activation:
    - batch\_size context\_length d\_model -> batch\_size context\_length d\_model
    - $B \times T \times d$
- RMSNorm(ln2)
  - Parameter:  $d$
  - Activation:
    - rms: batch\_size context\_length 1
      - $B \times T$
    - output: batch\_size context\_length d\_model
      - $B \times T \times d$
- SwiGLU(ffn)
  - Parameter( $W_1, W_2, W_3$ ):  $3 \times d \times d_{\text{ff}}$
  - Activation:
    - $W_1$  projection:
      - batch\_size context\_length d\_model -> batch\_size context\_length d\_ff
      - $B \times T \times d_{\text{ff}}$
    - $W_3$  projection:
      - batch\_size context\_length d\_model -> batch\_size context\_length d\_ff
      - $B \times T \times d_{\text{ff}}$
    - SiLU:
      - batch\_size context\_length d\_ff
      - $B \times T \times d_{\text{ff}}$
    - element-wise product:
      - batch\_size context\_length d\_ff
      - $B \times T \times d_{\text{ff}}$
    - $W_2$  projection:
      - batch\_size context\_length d\_ff -> batch\_size context\_length d\_model
      - $B \times T \times d_{\text{model}}$
- RMSNorm(ln\_final)
  - Parameter:  $d$
  - Activation:
    - rms: batch\_size context\_length 1
      - $B \times T$
    - output: batch\_size context\_length d\_model
      - $B \times T \times d$

- Linear(lm\_head)
  - Parameter:  $d \times V$
  - Activation:
    - batch\_size context\_length d\_model  $\rightarrow$  batch\_size context\_length vocab\_size
    - $B \times T \times V$
- cross-entropy on logits
  - Activation:  $B \times T$

Elements:

- Parameter:  $d \times (1 + 2 \times V + 2 \times L + 12 \times L \times d)$
- Activation:  $\frac{1}{3} \times B \times T \times (6 + 3 \times V + 6 \times T \times L \times H + 6 \times L + 56 \times L \times d + 3 \times d)$
- Gradients:  $d \times (1 + 2 \times V + 2 \times L + 12 \times L \times d)$
- Optimizer state:  $2 \times d \times (1 + 2 \times V + 2 \times L + 12 \times L \times d)$
- Total:  $\frac{1}{3} \times (3 \times B \times V \times T + 6 \times B \times T + 6 \times B \times T \times L + 56 \times B \times T \times L \times d + 3 \times B \times T \times d + 6 \times B \times T^2 \times L \times H + 24 \times V \times d + 24 \times L \times d + 144 \times L \times d^2 + 12 \times d)$

Bytes:

- Parameter:  $4 \times d \times (1 + 2 \times V + 2 \times L + 12 \times L \times d)$
- Activation:  $\frac{4}{3} \times B \times T \times (6 + 3 \times V + 6 \times T \times L \times H + 6 \times L + 56 \times L \times d + 3 \times d)$
- Gradients:  $4 \times d \times (1 + 2 \times V + 2 \times L + 12 \times L \times d)$
- Optimizer state:  $8 \times d \times (1 + 2 \times V + 2 \times L + 12 \times L \times d)$
- Total:  $\frac{4}{3} \times (3 \times B \times V \times T + 6 \times B \times T + 6 \times B \times T \times L + 56 \times B \times T \times L \times d + 3 \times B \times T \times d + 6 \times B \times T^2 \times L \times H + 24 \times V \times d + 24 \times L \times d + 144 \times L \times d^2 + 12 \times d)$

(b) 此处使用 GPT-2 XL 的实际对齐值  $d_{\text{ff}} = 4288$

In GPT-2 XL-shaped model:

- Parameters(1,640,452,800 elements): 6,561,811,200 bytes.
- Activations(4,041,985,024  $\times B$  elements) : 16,167,940,096B bytes.
- Gradients(1,640,452,800 elements): 6,561,811,200 bytes.
- Optimizer states(3,280,905,600 elements): 13,123,622,400 bytes.
- Total(4,041,985,024  $\times B$  + 6,561,811,200 elements): 16,167,940,096B + 26,247,244,800 bytes (about 15.06B + 24.44 GiB)

可以适配的最大 batch size 是 3

(c) 记总参数数量为  $P$

- Apply weight decay:  $2P$
- Update the first moment estimate:  $3P$
- Update the second moment estimate:  $4P$
- Apply moment-adjusted weight updates:  $5P$

所以总 FLOP 为  $14P$

(d) Nvidia H100 GPU peak = 495 teraFLOP/s =  $495 \times 10^{12}$  FLOP/s

observed throughput = MFU  $\times$  peak = 50%  $\times 495 \times 10^{12}$  FLOP/s

forward total FLOPs =  $400,000 \times 1,024 \times$

$3,516,769,894,400 = 1,440,468,948,746,240,000$  FLOPs

backward total FLOPs =  $2 \times 400,000 \times 1,024 \times$

$3,516,769,894,400 = 2,880,937,897,492,480,000$  FLOPs

AdamW total FLOPs =  $400,000 \times 14 \times P = 400,000 \times 14 \times 1,640,452,800 = 9,186,535,680,000,000$  FLOPs

time =  $\frac{\text{forward total FLOPs} + \text{backward total FLOPs} + \text{AdamW total FLOPs}}{\text{observed throughput}} = \frac{4,321,416,032,774,400,000,000}{50\% \times 495 \times 10^{12}} \approx 17,460,267 \text{ s} \approx 4,850 \text{ hours}$

## 9. Problem (learning\_rate): Tune the learning rate (2 B200 hrs) (3 points)

### Prompt

The learning rate is one of the most important hyperparameters to tune. Taking the base model you've trained, answer the following questions.

(a) Perform a hyperparameter sweep over the learning rates and report the final losses, or note divergence if the optimizer diverges.

**Deliverable:** Learning curves associated with multiple learning rates. Explain your hyperparameter search strategy.

**Deliverable:** A model with validation loss, per token, on TinyStories of at most 1.45.

Low-Resource Tip: Train for a few steps on CPU or Apple Silicon

If you are running on `cpu` or `mps`, you should instead reduce the total tokens processed count to 40,000,000, which will be sufficient to produce reasonably fluent text. You may also increase the target validation loss from 1.45 to 2.00.

Running our solution code with a tuned learning rate on an M4 Max chip and 36 GB of RAM, we use `batch size × total step count × context length = 32 × 5000 × 256 = 40,960,000` tokens, which takes 1 hour and 22 minutes on `cpu` and 36 minutes on `mps`. At step 5000, we achieve a validation loss of 1.80.

Some Additional Tips

- When using `N` training steps, we suggest adjusting the cosine learning rate decay schedule to terminate its decay, that is, reach the minimum learning rate, at precisely step `N`.
- When using `mps`, do not use TF32 kernels. In particular, do not set:

```
torch.set_float32_matmul_precision("high")
```

as you might with CUDA devices. We tried enabling TF32 kernels with `mps`, using `torch` version 2.9.0, and found the backend sometimes uses silently broken kernels that cause unstable training.

- You can speed up training by JIT-compiling your model with `torch.compile`.

Specifically:

```
# On cpu
model = torch.compile(model)
```

```
# On mps
model = torch.compile(model, backend="aot_eager")
```

Compilation with Inductor is not supported on `mps` as of `torch` version 2.9.0.

(b) Folk wisdom is that the best learning rate is “at the edge of stability.” Investigate how the point at which learning rates diverge is related to your best learning rate.

**Deliverable:** Learning curves of increasing learning rate which include at least one divergent run and an analysis of how this relates to convergence rates.

## Answer

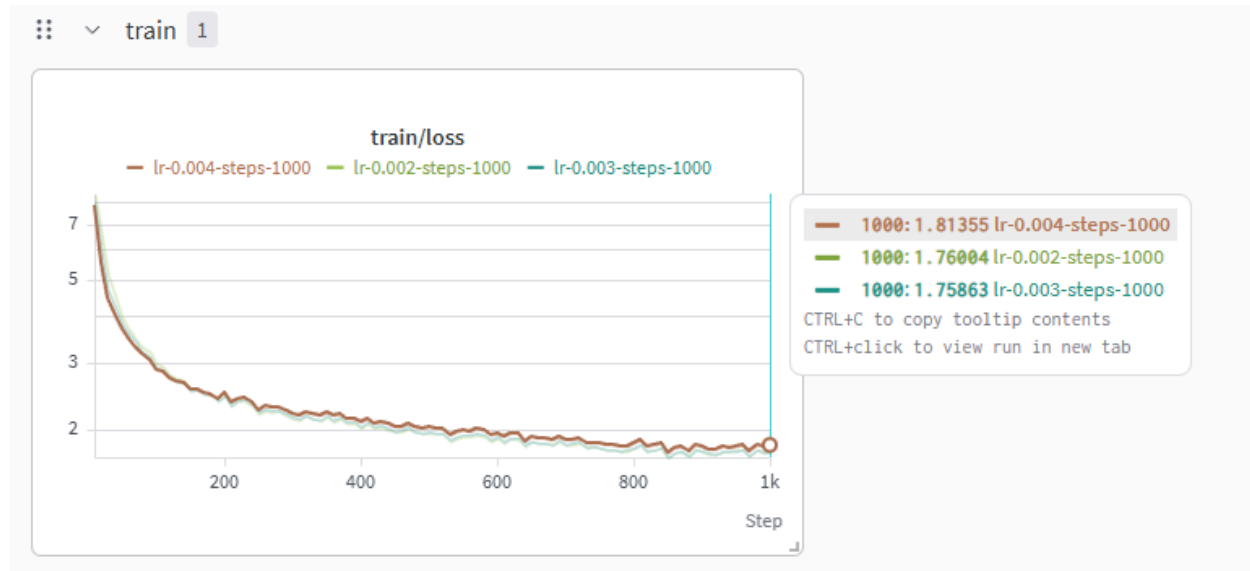
(a) 为了确定合适的学习率，我首先进行了一个粗粒度搜索。固定模型结构、batch size、优化器等其他超参数，仅改变 AdamW 的初始最大学习率，分别测试： $10^{-4}$ ,  $3 \times 10^{-4}$ ,  $10^{-3}$ ,  $3 \times 10^{-3}$ ,  $10^{-2}$ ，每组实验训练 1000 steps。

粗搜索结果如下：



| 学习率                | Train Loss | Validation Loss |
|--------------------|------------|-----------------|
| $10^{-4}$          | 2.8791     | 2.7990          |
| $3 \times 10^{-4}$ | 2.3151     | 2.2591          |
| $10^{-3}$          | 1.8797     | 1.8376          |
| $3 \times 10^{-3}$ | 1.7586     | 1.7287          |
| $10^{-2}$          | 2.7169     | 2.6335          |

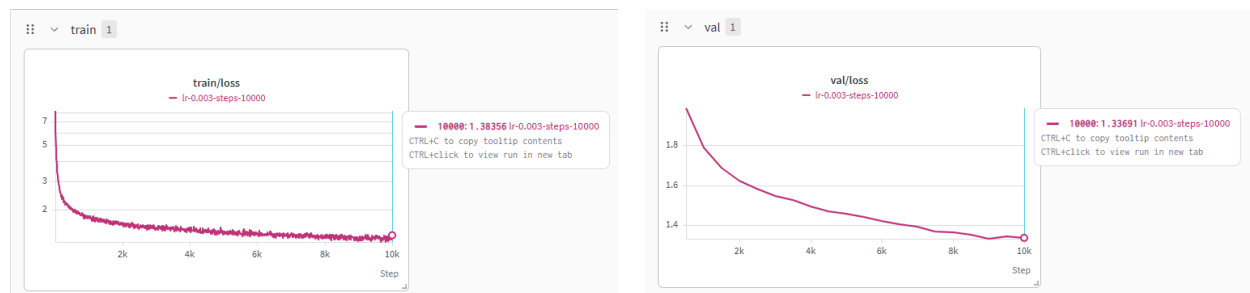
在 Step = 1000 下对  $2 \times 10^{-3}$ ,  $4 \times 10^{-3}$  训练



| 学习率                | Train Loss | Validation Loss |
|--------------------|------------|-----------------|
| $2 \times 10^{-3}$ | 1.7600     | 1.7299          |
| $3 \times 10^{-3}$ | 1.7586     | 1.7287          |
| $4 \times 10^{-3}$ | 1.8136     | 1.7780          |

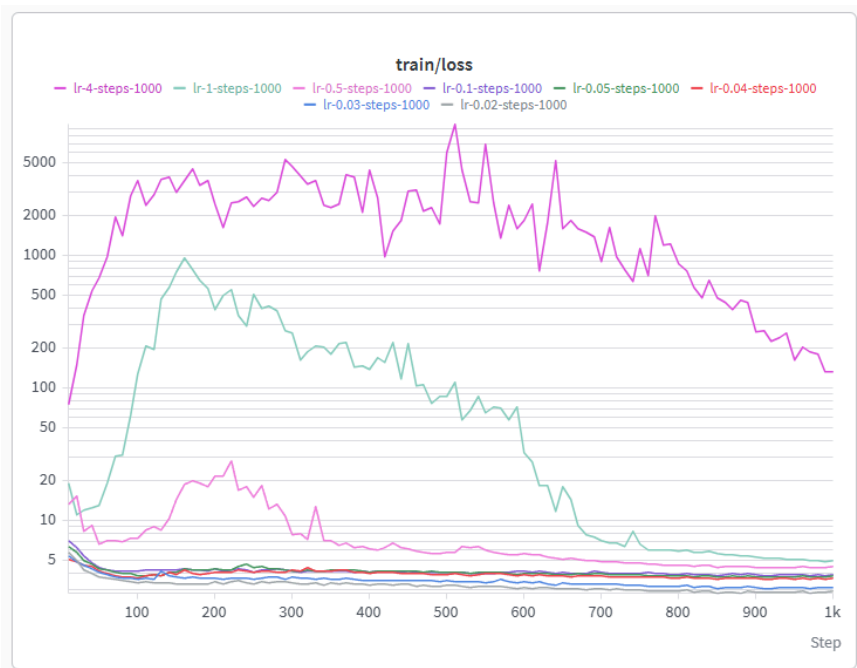
在这组实验中， $3 \times 10^{-3}$  仍然取得了最低的 Train Loss 和 Validation Loss。因此，在当前实验预算下选择  $3 \times 10^{-3}$  作为后续完整训练的学习率。

最终训练得到的 train\_loss 为 1.3836，val\_loss 为 1.3369，



(b)从大于 $10^{-2}$ 的 lr 开始寻找稳定边界。固定训练 1000 steps，逐渐增大学习率，观察 loss 曲线变化。首先测试较大的学习率：

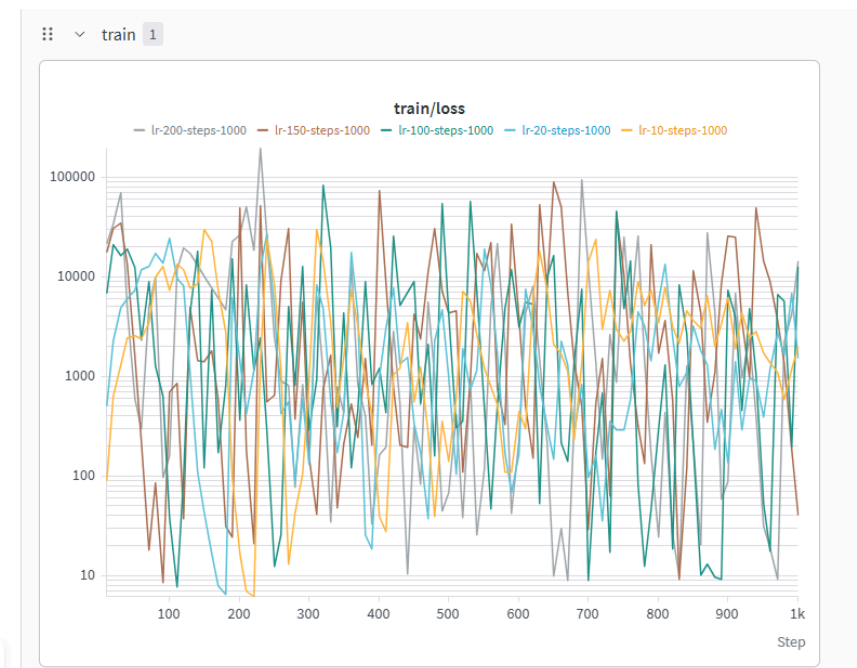
- lr 从 0.02 到 4



可以看到，虽然这些学习率相比最优学习率  $3 \times 10^{-3}$  已经明显增大，但部分实验仍然能够下降，因此仅观察最终是否 NaN 并不能准确刻画稳定边界。

进一步增大学习率：

- lr 从 10 到 200



可以观察到 loss 开始出现明显震荡，训练过程不再表现为稳定下降，但仍未立即产生 NaN。

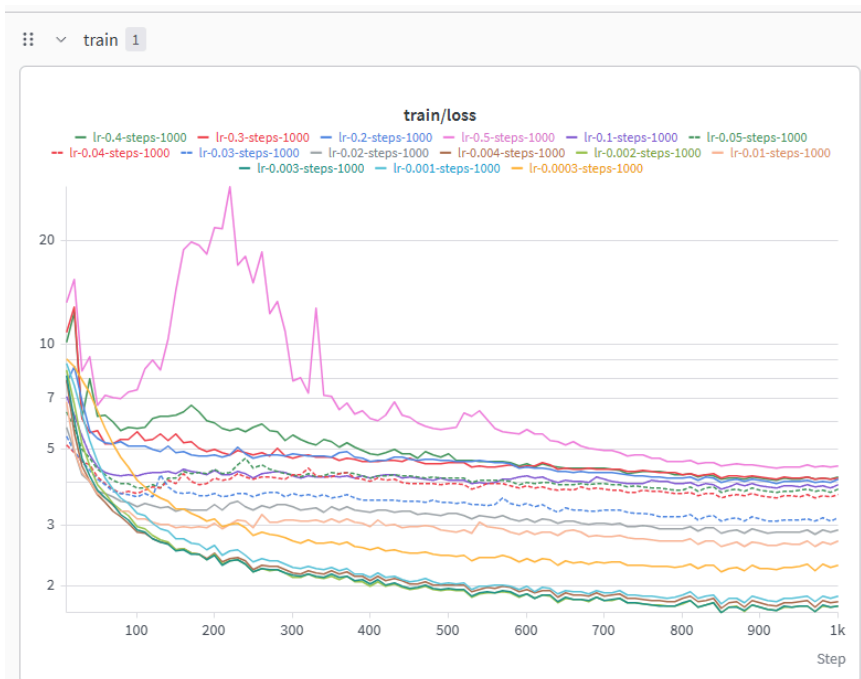
当学习率继续增大到300 以上后



训练出现 NaN，说明已经进入严重发散区域。

但是 NaN 并不是 stability edge 的位置。因为在这之前已经有很多训练是无法 stable 的了  
在较小范围内进一步观察：

- lr 从 0.001 到 0.5



可以看到：

- $lr \leq 4 \times 10^{-3}$  时，loss 持续下降；
- $lr \approx 10^{-2}$  附近开始出现较高 loss plateau；

- $lr = 0.02 \sim 0.05$  时出现明显震荡。

更大的学习率导致更强的不稳定。

这说明最佳学习率并不等于稳定边界，但位于其附近的稳定区域内。该结果与 “best learning rate is often close to the edge of stability” 的经验规律一致。

## 10. Problem (`batch_size_experiment`): Batch size variations (1 B200 hr) (1 point)

### Prompt

Vary your batch size all the way from 1 to the GPU memory limit. Try at least a few batch sizes in between, including typical sizes like 64 and 128.

**Deliverable:** Learning curves for runs with different batch sizes. The learning rates should be optimized again if necessary.

**Deliverable:** A few sentences discussing your findings on batch sizes and their impacts on training.

### Answer

TODO: Write your response here.

## 11. Problem (`generate`): Generate text (1 point)

### Prompt

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters, `temperature`, `top-p`, and so on, to get fluent outputs.

**Deliverable:** Text dump of at least 256 tokens of text, or until the first `<|endoftext|>` token, and a brief comment on the fluency of this output and at least two factors which affect how good or bad this output is.

### Answer

TODO: Write your response here.

## 12. Problem (`layer_norm_ablation`): Remove RMSNorm and train (0.5 B200 hrs) (1 point)

### Prompt

Remove all of the RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability by using a lower learning rate?

**Deliverable:** A learning curve for when you remove RMSNorms and train, as well as a learning curve for the best learning rate.

**Deliverable:** A few sentences of commentary on the impact of RMSNorm.

Answer

TODO: Write your response here.

### 13. Problem (`pre_norm_ablation`): Implement post-norm and train (0.5 B200 hrs) (1 point)

**Prompt**

Modify your pre-norm Transformer implementation into a post-norm one. Train with the post-norm model and see what happens.

**Deliverable:** A learning curve for a post-norm Transformer, compared to the pre-norm one.

Answer

TODO: Write your response here.

### 14. Problem (`no_pos_emb`): Implement NoPE (0.5 B200 hrs) (1 point)

**Prompt**

Modify your Transformer implementation with RoPE to remove the position embedding information entirely, and see what happens.

**Deliverable:** A learning curve comparing the performance of RoPE and NoPE.

Answer

TODO: Write your response here.

### 15. Problem (`swiglu_ablation`): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point)

**Prompt**

**Deliverable:** A learning curve comparing the performance of SwiGLU and SiLU feed-forward networks, with approximately matched parameter counts.

**Deliverable:** A few sentences discussing your findings.

Answer

TODO: Write your response here.

## 16. Problem (`main_experiment`): Experiment on OWT (2 B200 hrs) (2 points)

### Prompt

Train your language model on OpenWebText with the same model architecture and total training iterations as TinyStories. You may need to re-tune hyperparameters such as learning rate or batch size.

**Deliverable:** A learning curve of your language model on OpenWebText. Describe the difference in losses from TinyStories, and how those losses should be interpreted.

**Deliverable:** Generated text from the OpenWebText language model, in the same format as the TinyStories outputs. Comment on the fluency of this text and why the output quality is worse even with the same model and compute budget as TinyStories.

### Answer

TODO: Write your response here.

## 17. Problem (`leaderboard`): Leaderboard (10 B200 hrs) (6 points)

### Prompt

Train a model under the leaderboard rules from the handout, with the goal of minimizing validation loss within 0.75 B200-hours. The run must use only the provided OpenWebText training dataset and fit within a 45 minute wall-clock runtime on a B200.

**Deliverable:** The final validation loss, an associated learning curve with a wall-clock-time x-axis under 45 minutes, and a description of what you did.

### Answer

TODO: Write your response here.